

Lab 05 - Filtering, Enhancement and Restoration Measured with PSNR and SSIM

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 02: AI Image Generation, Editing and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K4	Image and video processing, filtering and transformation methods

Scenario

'Looks better' is not a deliverable. Take a clean reference frame, damage it three ways, repair it with five filters, and rank the repairs with PSNR and SSIM computed from the formulas - then examine whether the two metrics disagree and explain which one you would trust for a customer-facing asset. Classroom core (15 minutes): run the supplied enhancement comparison, inspect two contrasting results and explain why metrics alone cannot select the best creative output. The full implementation walkthrough and further parameter trials are extensions.

What you will produce

out/enhancement-matrix.csv scoring 5 filters against 3 degradations with PSNR and SSIM, plus a one-paragraph recommendation naming the winning filter per degradation and whether their rankings differ.

Tools: Python 3 | opencv-python | NumPy | offline with preinstalled dependencies; no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .env
source .env/bin/activate          # Windows: .env\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Load the clean reference frame
2. Apply three known degradations

3. Run five candidate filters on each
4. Score every result with PSNR and SSIM
5. Recommend a filter per degradation

Files in this folder

Path	What it is
reference/nq-hero-clean.png	Synthetic 800x600 clean reference frame with flat regions, hard edges, fine texture and a smooth gradient - the four content types that separate filters (deterministically rendered with OpenCV - SIMULATED)
data/degradations.json	The three degradations with their exact parameters and random seed
data/filter-bank.json	The five candidate filters with their kernel sizes and parameters
data/expected-ranking.csv	Reference winning filter per degradation, for self-verification
enhance.py	Runnable script - see the steps below
ssim.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Use the prepared offline environment and work inside this lab folder. For fragments use one notebook/REPL with `import cv2, numpy as np; img=cv2.imread('reference/nq-hero-clean.png');` `assert img is not None`. The full script saves each degraded/filtered image; select one pair as `a=img` and `b=cv2.imread('out/fixed_<degradation>_<filter>.png')` before PSNR comparison.
2. Load `reference/nq-hero-clean.png`. This is the ground truth: every metric in this lab is computed against it, so nothing is filtered before it is measured.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-hero-clean.png');print(i.shape)"
```

3. Read `data/degradations.json`. The three degradations are: GAUSSIAN noise (sigma 18), SALT-AND-PEPPER impulse noise (4% of pixels), and DEFOCUS blur (Gaussian, sigma 2.5). The random seed is fixed; allow small implementation-dependent float differences and report measured results.
4. Apply degradation 1 - additive Gaussian noise. Note that you must clip to [0,255] and cast back to uint8; casting out-of-range sums back to uint8 without clipping wraps around and produces speckle that is not the noise you intended.

```
rng = np.random.default_rng(20260906)
noise = rng.normal(0, 18, img.shape)
gauss = np.clip(img.astype(np.float64) + noise, 0, 255).astype(np.uint8)
```

5. Apply degradation 2 - salt and pepper. Choose 4% of pixel positions at random and set half to 0 and half to 255. This is impulse noise: a few pixels are catastrophically wrong while the rest are untouched. The supplied function

samples unique positions (rounded down to whole pixels).

```
from enhance import degrade_salt_pepper
impulse=degrade_salt_pepper(img,0.04,20260906)
```

6. Apply degradation 3 - defocus blur with `cv2.GaussianBlur(img, (0,0), 2.5)`. Passing (0,0) for ksize makes OpenCV derive the kernel size from sigma, which is what you want when sigma controls the simulated blur strength.

```
defocus=cv2.GaussianBlur(img, (0,0),2.5)
```

7. Now the five filters. Filter 1 - BOX blur, `cv2.blur(img, (5,5))`. This is a 5x5 kernel of 1/25, the simplest low-pass. Write out the kernel by hand and confirm it sums to 1; a kernel that does not sum to 1 changes the image brightness.

8. Filter 2 - GAUSSIAN blur, `cv2.GaussianBlur(img, (5,5), 0)`. Weights fall off with distance from the centre, so its smoothing differs from the box filter at the same support. With sigma=0 OpenCV derives sigma from ksize.

9. Filter 3 - MEDIAN, `cv2.medianBlur(img, 5)`. This is a rank filter, not a convolution: it replaces each pixel with the median of its neighbourhood. Predict now which degradation it will dominate, and why a median is more resistant to isolated outliers than a mean is not.

10. Filter 4 - BILATERAL, `cv2.bilateralFilter(img, 9, 75, 75)`. It weights neighbours by spatial distance AND by intensity difference, so it smooths within a region but not across an edge. It can be costly - time it and record the cost.

11. Filter 5 - UNSHARP MASK. Build it yourself: blur the image, then `cv2.addWeighted(img, 1.5, blurred, -0.5, 0)`. This is sharpening expressed as 'original plus the high-frequency residual', and it can amplify noise; any unsuitable filter can make a noisy image worse.

```
blur = cv2.GaussianBlur(img, (0, 0), 3)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
```

12. Compute PSNR from the formula before you use the library call. $MSE = \text{mean}((a-b)^2)$ over all pixels and channels; $PSNR = 10 * \log_{10}(255^2 / MSE)$. Verify your hand-rolled value against `cv2.PSNR` to within 0.01 dB.

```
mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
psnr = 10 * np.log10(255.0 ** 2 / mse)
print(psnr, cv2.PSNR(a, b))
```

13. Compute SSIM. The base `opencv-python` wheel does NOT ship `cv2.quality`, so the lab provides a NumPy implementation in `ssim.py`. Read it: SSIM combines luminance, contrast and structure terms as $((2*\mu_x*\mu_y + C1)(2*\sigma_{xy} + C2)) / ((\mu_x^2 + \mu_y^2 + C1)(\sigma_x^2 + \sigma_y^2 + C2))$ with $C1=(0.01*255)^2$ and $C2=(0.03*255)^2$, computed over a sliding Gaussian window.

14. Run the full 3x5 matrix. Every cell reports PSNR and SSIM of the filtered result against the clean reference, plus the filter's wall-clock time.

```
python3 enhance.py --clean reference/nq-hero-clean.png --degradations data/degradations.
json --filters data/filter-bank.json --out out/enhancement-matrix.csv
```

15. Read the matrix. Test the common hypothesis that MEDIAN performs well on salt-and-pepper; the ranking depends on the image and parameters. BILATERAL may preserve edges in Gaussian noise, but these filters cannot guarantee recovering detail lost to blur; deblurring is an inverse problem rather than just smoothing one.

16. Find the disagreement. Check whether PSNR and SSIM rank two filters differently; report agreement if none differ. For a disagreement explain: PSNR reflects mean squared error and may favour aggressive smoothing; SSIM is a windowed structural comparison that may penalise lost texture that the smoothing cost you.

17. Decide which metric you would gate a customer-facing asset on, and write one paragraph justifying it. Note the wider point for generated media: distributional metrics such as FID and perceptual metrics such as LPIPS exist for exactly this reason - a single per-pixel number does not describe perceived quality.

18. Compare your winners against data/expected-ranking.csv, save out/enhancement-matrix.csv and your recommendation paragraph, and complete the evidence checklist.

```
python3 -c "import csv;print(open('data/expected-ranking.csv').read())"
```

Verify

out/enhancement-matrix.csv has 18 rows (15 filtered comparisons plus 3 baselines) each with a PSNR and an SSIM, your hand-computed PSNR agrees with cv2.PSNR to 0.01 dB, MEDIAN has a reported rank on salt-and-pepper, and you have checked whether PSNR and SSIM disagree.

Expected outputs

- out/enhancement-matrix.csv - 18 rows: degradation, filter, psnr_db, ssim, ms.
- Baseline row per degradation (no filter) so every improvement is measured against it.
- Observed MEDIAN rank on salt-and-pepper, with evidence.
- Observed BILATERAL rank on Gaussian noise.
- Measured improvement or worsening relative to defocus baseline; no recovery guarantee.
- Measured UNSHARP change relative to each noisy baseline.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ out/enhancement-matrix.csv pasted in full.
- ☐ Your hand-computed PSNR for one cell, next to the cv2.PSNR value.
- ☐ The 5x5 box kernel written out, with its sum.
- ☐ Rankings by both metrics, including any disagreement or a statement of agreement.
- ☐ One paragraph recommending a metric to gate customer-facing assets, with reasoning.
- ☐ The bilateral filter's wall-clock time compared with the box filter's.

Troubleshooting

Symptom	What to do
PSNR is inf	MSE is zero: inputs are identical, whether intentionally or by an input mistake. Check you are passing the filtered result, not the source.
Adding noise makes the image look like static confetti	Adding floats promotes the array; casting out-of-range results back to uint8 can wrap values. Cast to float64, add, np.clip(..., 0, 255), then cast back to uint8.
cv2.medianBlur raises an error	ksize must be odd and greater than 1, and for ksize > 5 the input must be uint8. Use 3, 5 or 7.
cv2.quality is not available	Correct - the SSIM module lives in opencv-contrib-python, not in the base wheel. This lab therefore ships its own NumPy SSIM in ssim.py; use that. Do not add a contrib dependency for one function.

Symptom	What to do
SSIM values look far too high for a badly damaged image	Check inputs, alignment, dynamic range and pooling; a high score alone does not establish the cause. Convert both images to grayscale first, or average the per-channel SSIM maps - <code>ssim.py</code> does the latter.
The bilateral filter takes several seconds	That is the finding, not a fault. Record the time; it is the argument for running restoration at the edge only when the latency budget allows it.

References used in this lab

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD², LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: `cv2.data.haarcascades` ships the frontal-face and eye cascades so no weights download is required; `cv2.HOGDescriptor_getDefaultPeopleDetector()` is built in; `SIFT_create`, `ORB_create`, `FastFeatureDetector_create`, `cornerHarris`, `goodFeaturesToTrack`, `matchTemplate`, `createBackgroundSubtractorMOG2/KNN`, `meanShift`, `CamShift`, `calcOpticalFlowFarneback` and `PSNR` are all present. IMPORTANT CORRECTION carried into the labs: `cv2.TrackerCSRT_create` and `cv2.TrackerKCF_create` are NOT in the base `opencv-python` wheel (they require `opencv-contrib-python`), and the `cv2.quality SSIM` module is also contrib-only - so the labs use `cv2.TrackerMIL_create` and a NumPy SSIM implementation to stay dependency-light and runnable offline.

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11*.

Historical evolution of synthetic media; role of data in training generative models; overview of major architectures; text-to-image, style transfer and super-resolution as distinct image-synthesis tasks; frame-based versus temporal video generation, motion transfer, and the temporal-consistency and realism challenges.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.